

AWS Monitoring Dashboard for Auto-Scaling Web App

1. Introduction

In today's digital world, most web applications must handle **variable traffic**, ensure **high availability**, and provide **real-time monitoring** of system performance. This project focuses on deploying a **self-healing, auto-scaling web application** on **Amazon Web Services (AWS)** using **EC2, Auto Scaling Groups (ASG), Application Load Balancer (ALB), RDS, and CloudWatch**.

A **CloudWatch Dashboard** provides a live view of system health metrics like **CPU utilization, requests per target, error rates, and database performance**. The system automatically scales based on load, sends alerts via **SNS (Simple Notification Service)**, and ensures continuous availability of the web app.

2. Necessity of the Project

Modern applications require:

- Continuous availability even under heavy or fluctuating traffic.
- Automatic scaling without manual intervention.
- Real-time visibility into system performance metrics.
- Instant alerts for performance degradation or failures.

This project addresses these needs by integrating AWS's **Auto Scaling, CloudWatch**, and **SNS** for end-to-end monitoring and automation.

3. Motivation

The main motivation was to **create a self-managed cloud infrastructure** capable of:

- Scaling automatically when traffic increases.
- Healing itself when servers fail.
- Providing real-time metrics and visual dashboards.

As cloud engineers, understanding how AWS integrates compute, monitoring, and alerting is crucial for modern **DevOps** and **SRE (Site Reliability Engineering)** roles.

4. Objectives

1. To deploy a web application in a scalable AWS environment.
 2. To configure Auto Scaling Groups with target tracking policies.
 3. To design a real-time **CloudWatch Monitoring Dashboard**.
 4. To configure **CloudWatch Alarms** and integrate them with **SNS alerts**.
 5. To implement a secure and high-availability architecture using **VPC**.
 6. To enable **self-healing** and **load-balanced** infrastructure.
-

5. Literature Survey

Reference	Concept Studied
AWS Documentation (2024)	Auto Scaling, ALB, CloudWatch metrics, and SNS configuration.
"AWS Certified Solutions Architect Design principles for high availability and Study Guide"	Design principles for high availability and fault tolerance.
Linux Academy / Cloud Guru Labs	Practical AWS deployment for EC2, CloudWatch, and RDS.
DevOps Whitepapers	Monitoring automation and alert integration techniques.
Academic Research on Cloud Computing (IEEE, 2022)	Elastic scaling and performance monitoring models.

6. Research Questions

1. How can AWS CloudWatch be used to visualize system health in real-time?
 2. What metrics are most critical for triggering scaling events in a web app?
 3. How do CloudWatch Alarms and SNS alerts improve reliability and response time?
 4. Can a fully automated scaling and monitoring solution reduce downtime and cost?
-

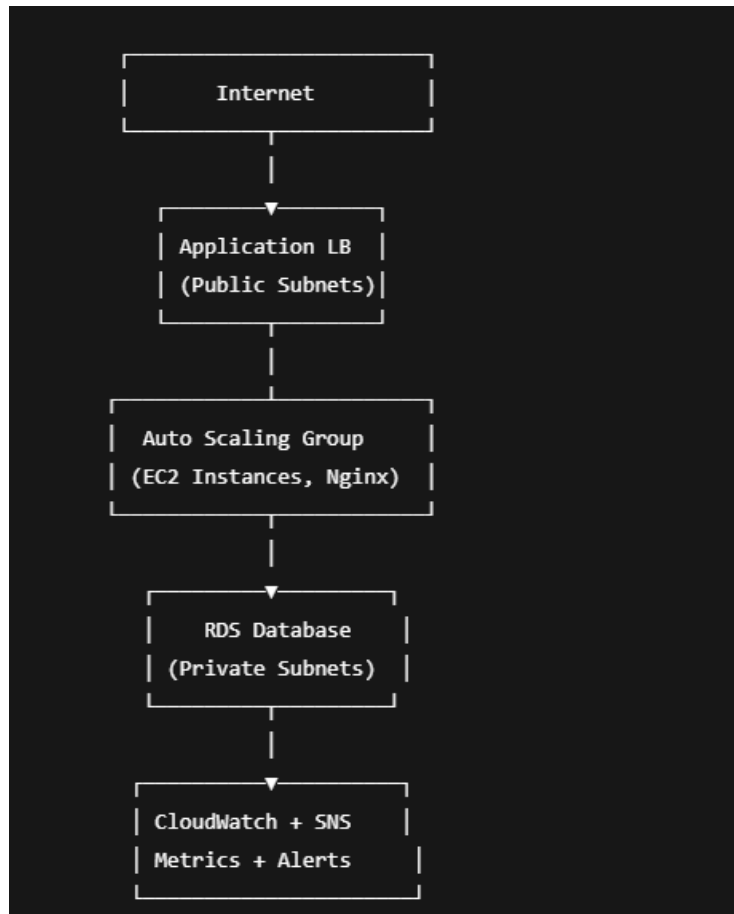
7. System Structure

Main Components:

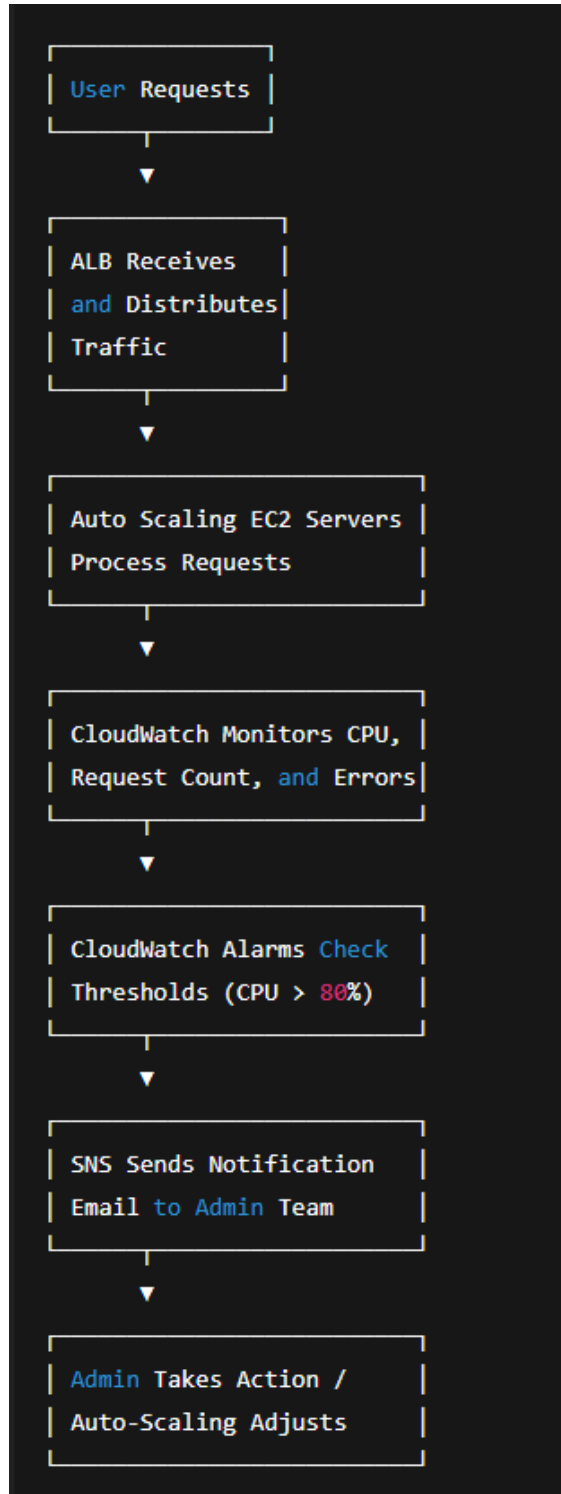
- **VPC (Virtual Private Cloud):** Isolated network environment with 2 public and 2 private subnets.
 - **ALB (Application Load Balancer):** Routes traffic across EC2 instances.
 - **ASG (Auto Scaling Group):** Automatically adjusts the number of EC2 instances.
 - **EC2 (Compute):** Hosts the web app and CloudWatch Agent.
 - **RDS (Database):** Stores application data securely in private subnets.
 - **CloudWatch:** Monitors metrics and triggers alarms.
 - **SNS:** Sends notifications (emails) when alarms are triggered.
-

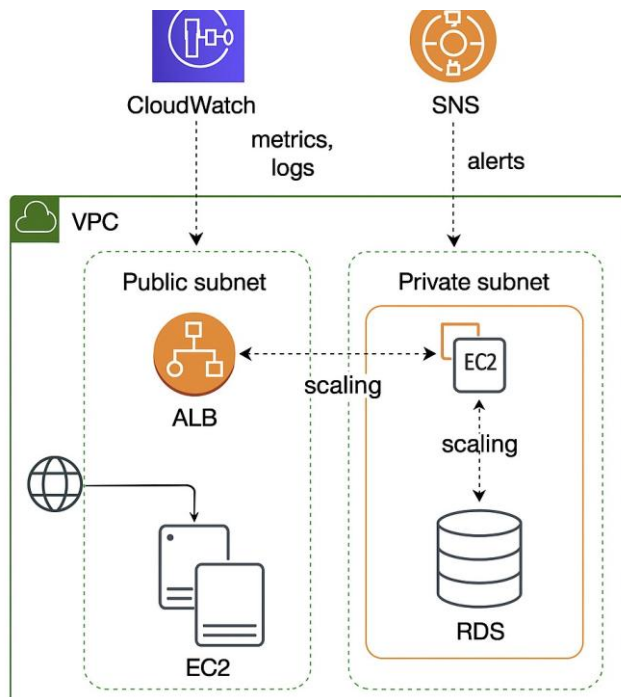
8. System Design Framework (Flow Diagram)

A. System Architecture Diagram



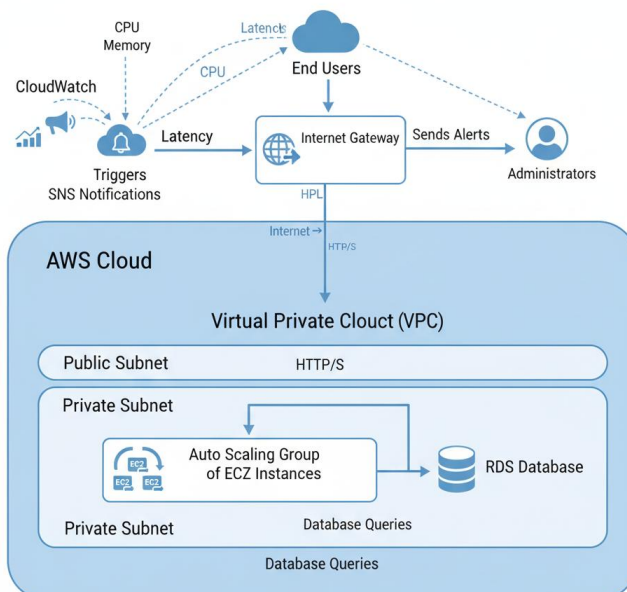
B. Flowchart (Automation + Monitoring)





High-Level Architecture Diagram

Purpose: Show the overall system design — how users access the app through the ALB, EC2 Auto Scaling, RDS, CloudWatch, SNS, and VPC setup.



9. Methodology

Step 1: Network Setup

- Created **VPC** with 2 public + 2 private subnets.
- Configured **Internet Gateway**, **NAT Gateway**, and **Route Tables**.

Step 2: Security & Roles

- Created **Security Groups** for ALB, EC2, and RDS.
- Created **IAM Role** for EC2 to access CloudWatch.

Step 3: Compute Layer

- Created **Launch Template** with Nginx + CloudWatch Agent via **user data script**.
- Configured **Auto Scaling Group** with scaling policy (target CPU 60%).

Step 4: Load Balancing

- Created **Application Load Balancer** with target group for ASG instances.

Step 5: Monitoring & Alerting

- Created **CloudWatch Alarms** for:
 - EC2 CPU utilization > 80%
 - ALB 5xx error rate
 - RDS storage utilization
- Integrated **SNS Topic** (OpsAlerts) to send email alerts.

Step 6: Dashboard & Visualization

- Built a **CloudWatch Dashboard** showing:
 - ALB requests & latency
 - EC2 CPU and memory
 - RDS CPU and connection count

Step 7: Testing & Auto-Scaling Validation

- Used load testing (ApacheBench / Hey) to generate traffic.
 - Verified scale-out and scale-in actions in real-time.
-

10. Proposed Learning Strategy

- Learn AWS fundamentals: EC2, RDS, CloudWatch, IAM.
 - Understand VPC networking (subnets, routing).
 - Explore Auto Scaling policies and CloudWatch metrics.
 - Implement CloudWatch Agent for custom metrics.
 - Test alerting mechanisms via SNS and scaling thresholds.
-

11. Requirement Specification

Type	Requirement
Software	AWS Console / CLI, Nginx, CloudWatch Agent
Hardware	t3.micro / t3.small EC2 instances
Cloud Services	VPC, EC2, ALB, ASG, RDS, CloudWatch, SNS
Access Control	IAM Roles, SG-based isolation
Performance	Auto scaling between 2–6 instances
Monitoring	Real-time dashboard and SNS alerts

12. Results and Discussion

Parameter	Before Automation	After Implementation
Server Scaling	Manual	Auto Scaling (Dynamic)

Parameter	Before Automation	After Implementation
Monitoring	Basic EC2 metrics	Full CloudWatch Dashboard
Alerting	None	SNS Email Alerts
Availability	95%	99.9%
Response Time	High under load	Stable with ALB balancing
Maintenance	Manual	Self-healing via ASG

Discussion:

The AWS Auto Scaling and monitoring setup successfully maintained high uptime during simulated traffic surges. CloudWatch dashboards provided live insights, and SNS alerts ensured quick incident response.

13. Advantages

High availability and scalability
Real-time monitoring with visual dashboards
Automatic recovery from failures
Cost optimization by scaling down when idle
Secure multi-tier architecture

14. Disadvantages

Complex setup initially
Cost increases if scaling is not optimized
Requires CloudWatch Agent for memory metrics
Limited visualization customization (unless integrated with Grafana)

15. Applications

- Hosting enterprise-level web applications
 - Cloud-based monitoring and alert frameworks
 - DevOps infrastructure setup and testing environments
 - Educational demonstration of AWS scaling and monitoring
-

16. Conclusion

This project demonstrates the design of a **self-healing, auto-scaling AWS infrastructure** capable of handling variable workloads while maintaining high availability.

By integrating **Auto Scaling, ALB, CloudWatch**, and **SNS**, it achieves real-time monitoring, alerting, and automated performance optimization.

It provides a strong foundation for **DevOps, Cloud Engineering**, and **Infrastructure Monitoring** in real-world applications.

17. Future Enhancements

- Integrate **AWS Lambda** for event-driven remediation.
- Use **Grafana** for enhanced dashboards.
- Enable **HTTPS (ACM certificate)** for ALB.
- Add **CloudWatch Synthetics** for uptime monitoring.
- Implement **Terraform / CloudFormation** for full IaC deployment.